EASY RENTALS STUDENT ACCOMODATION MANAGEMENT SYSTEM

Software Requirements Specification

Team Members

221030472 Thabethe SB (Project Manager)

223035662 Magwai T

223051300 Mahase T

224061529 Sithole TS

223035639 Lesekele P

223059551 Goitsemodimo T

224076426 Shea OG

221000945 Malachamela T

Table of Contents

Project Title 3
Problem Statement 3
Purpose 3
Project Aim 4
Project Objective 4
Project Deliverables 4
Acceptance criteria 4
Project Exclusions 4
Project Constraints 4
Project Assumption 4
Definitions and Abbreviations 4
Stakeholders and user groups 5
Current process and diagram 5
Product perspective 6
Main Product Functions 7
Functional Requirements 7
Non-Functional Requirements 7
System Requirements 8
Integration Requirements 8
PRIORITISE THE REQUIREMENTS 16
ADD SUPPORTING EVIDENCE 20
Microsoft Project Schedule 22
Github Repository Link 23
Supabase Project 23
Designing System Architecture Main Components 24
Identifying the Actors 28
Participants of the ASP.NET sequence diagram 29
Participants of the Flutter sequence diagram 31
Review of Approved Requirements 33
Identification of ASP.NET Screens 35
Mobile and Web Integration 39
Design against the ERD 39
1. Flutter Mobile App (Prospective Student) 39
2. ASP.NET Web Portal (Easy Rentals Staff) 41

Alignment across the System Design.....42
Updated Gantt Chart.....47
Git Hub Repository.....47
Questions Posed to the stakeholder 47
Stakeholder Feedback Evidence 48

Phase 1

Project Title

Easy Rentals Student Accommodation Management System

Problem Statement

The organization currently uses a process of handling one student at a time to view student accommodation. This results in high petrol/travel cost, inconvenience between students and stakeholders/current tenants/landlord, administrative burden, and repeated individual viewings. The problem affects Easy Rentals(landlord) because the current process results in wasted and additional administrative work, as landlord must repeatedly conduct individual viewings. The problem also affects prospective students as they may have to pay transportation costs to travel and view accommodation before they have confirmed whether they will rent the property and this creates unnecessary costs and inconvenience. An improved solution is need to make the accommodation process more efficient and reduce unnecessary costs and administrative work.

Purpose

The proposed project prototype is needed to improve the way Easy Rentals:

- Provides accommodation information.
- Allows prospective students to review properties.
- Manages student enquiries.
- Manages accommodation/property viewings.
- Manages viewing arrangements.

The system is expected to improve:

- Time management.
- Communication between students/users and landlords.
- Management of viewing arrangements.
- Administrative workload.
- Costs associated with repeated enquiries and physical viewings.

Project Aim

Develop an integrated prototype that improves the student accommodation enquiry and viewing process.

Project Objective

Providing prospective students with accommodation information digitally and improving the management of accommodation enquiries and viewing arrangements.

Project Deliverables

Flutter Mobile application, ASP.NET web application, Supabase backend, project documentation and requirements documentation.

Acceptance criteria

Student can access information through Flutter mobile application and Web application. The mobile and web application should be sync with the shared Supabase backend.

Project Exclusions

Full rental life cycle management, rental collection, maintenance management, payment management and other processes outside property management.

Project Constraints

Complete within the time allocation, keep the Project within the project scope and technological constraints.

Project Assumption

After review of the first prototype, the stakeholder might suggest a few changes according to his requirements or specifications.

Definitions and Abbreviations

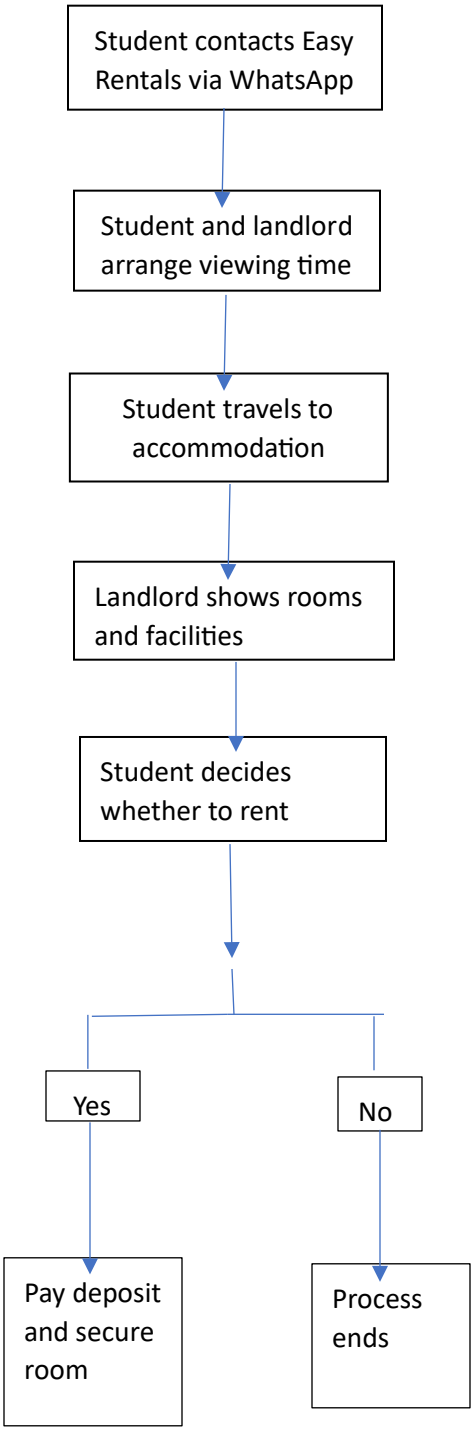
Term or abbreviation	Meaning
SRS	Software Requirements Specification
Flutter	Framework used to develop the mobile application
ASP.NET	Framework used to develop the web application
Supabase	Shared online backend and database

Stakeholders and user groups

Stakeholders or user	Role in the proposed system
External Stakeholder	Easy Rentals provides requirements and uses the system to manage accommodation and enquires
Mobile app user	Prospective student to view info and enquire.
Web app user	Easy rental to manage relevant property and enquire/view info
Other affected users	Current tenants they may benefit indirectly from unnecessary viewings

Current process and diagram

The current process starts when a student contacts Easy Rentals through WhatsApp to enquire about accommodation and request a viewing. The student and the landlord then communicate individually to arrange a suitable viewing time. The landlord has to coordinate a time that works for the student and then meet the student at the accommodation. During the viewing, the landlord takes the student through the accommodation and shows the available rooms and facilities. The student receives information about the property and decides whether the accommodation is suitable. If the student decides to take the room, they pay a deposit to secure the room.



Product perspective

The proposed project will include a flutter mobile app for prospective student and an ASP.NET web app is for Easy Rentals staff. Both app will connect to same Supabase Backend and share appointment info or feedback information.

Main Product Functions

Component	Main functions
Flutter mobile application	Sign up, Sign-in, view accommodation information, view property details, make appointment, check appointment status, send feedback and enquiries.
ASP.NET web application	Sign-in, manage accommodation information, view and manage prospective students enquiries, manage appointment/viewingrelated information, view student information
Supabase backend	Stores user profile accounts, stores property information, stores property enquiries and feedback, stores appointment information and enables data sharing between flutter and ASP.NET.

Functional Requirements

- FR1: The system must allow prospective students to view available accommodation information.
- FR2: The system must allow prospective students to view property details, including the property address, number of rooms, accommodation rules, deposit price, available facilities and available/moving-in date
- FR3: The mobile application must allow prospective students to submit an accommodation enquiry.
- FR4: The mobile application must allow prospective students to request an accommodation viewing.
- FR5: The web application must allow Easy Rentals to view prospective student enquiries.
- FR6: The web application must allow Easy Rentals to manage accommodation/property information.
- FR7: The web application must allow Easy Rentals to view and manage accommodation viewing requests using basic date/time selection (simplified from full calendar scheduling following the Unit 2 feasibility study).
- FR8: The system must store relevant prospective student information required for the accommodation enquiry process.
- FR9: The system must allow relevant accommodation and enquiry information to be shared between the mobile and web applications.
- FR10: The system must store relevant accommodation information required by prospective students.
-

Non-Functional Requirements

- NFR1: Only authorized Easy Rentals users must be able to access or modify management information through the web application
- NFR2: The system must protect prospective student information from unauthorized access.

- NFR3: The mobile interface must be understandable and easy for prospective students to navigate.
- NFR4: Error messages displayed to users must be clear and understandable.
- NFR5: The prototype must use test/development data where real confidential information is not required.
- NFR6: The system must maintain consistent accommodation and enquiry information between the mobile and web applications.

System Requirements

- SR1: The mobile application must be developed using Flutter.
- SR2: The web application must be developed using ASP.NET.
- SR3: The shared backend/database must use Supabase.
- SR4: The project's source code and relevant documentation must be maintained using GitHub.
- SR5: The system must be developed and tested using appropriate development environments for Flutter, ASP.NET and Supabase.

Integration Requirements

- IR1: The Flutter mobile application must communicate with the Supabase backend to retrieve relevant accommodation information.
- IR2: The Flutter mobile application must send prospective student enquiries and viewing requests to the Supabase backend.
- IR3: The ASP.NET web application must retrieve relevant accommodation, enquiry and viewing information from Supabase.
- IR4: Information submitted through the mobile application must be available to authorized Easy Rentals users through the web application.
- IR5: Accommodation information updated through the web application must be reflected in the relevant information available through the mobile application.
- IR6: The mobile application and web application must use the Supabase backend as the shared source of relevant project data.

Feasibility Review

The findings below were taken from the Feasibility Study Report (Activity 2.1) and confirm whether each area of the project needs adjustment before development continues.

Technical Feasibility

Decision: Feasible with modifications.

The core features (property viewings, enquiries, and booking requests) are achievable with group's the group current flutter, ASP.NET and Supabase skills. The scheduling/viewing-request function must be

simplified and advanced calendar integration postponed to a later phase. Integration between the three components remains the biggest technical challenge and will need careful management.

Operational Feasibility

Decision: Feasible

The system directly improves Easy Rentals current manual and WhatsApp-based enquiry and viewing processes. It fits alongside the organization's existing workflow rather than replacing the full rental process. Basic user guide can be provided to Easy Rentals staff if needed.

Economic Feasibility

Decision: Feasible

All necessary resources (development computers, test devices, free tools, and Supabase free plan) are already available to the team at no extra cost. The system is also expected to cut down on Easy Rentals' administrative workload and reduce repeated travel and viewing-related costs.

Schedule Feasibility

Decision: Feasible with controlled scope.

The core system can be developed in the time remaining, but integration, testing and stakeholder feedback need dedicated time set aside. The project scope must be managed carefully and non-essential functions simplified, postponed or removed to ensure testing and fixes are not rushed or cut short.

Legal Feasibility

Decision: Feasible

Only student information necessary for accommodation enquiries will be collected, access will be restricted to unauthorized Easy Rentals users, and all credentials and private keys will be excluded from the GitHub repository.

Ethical Feasibility

Decision: Feasible

The stakeholder is fully involved and understands the project's purpose. No unnecessary personal information will be collected, and no negative impacts on prospective students or current tenants have been identified.

Feasibility Conclusion

Overall Decision: Feasible, with controlled scope and carefully management of technical integration, security, and schedule.

The project's main strengths are that it directly solves the accommodation enquiry and viewing process, clearly separates students-facing and staff-facing functions, uses a well defined Flutter/ ASP/NET/Supabase architecture, and has an engaged stakeholder. The main limitations are the

complexity of integrating the mobile and web apps via Supabase, the limited remaining time, the need for dedicated testing, and the possibility of further requirement changes from the stakeholder.

Findings and Required Changes

Finding	Does the project need to change	What change is required
Technical Feasibility	Yes	Simplify the viewing/appointment scheduling function (FR7) to basic date/time selection instead of full calendar integration; manage FlutterSupabase-ASP.NET integration and testing time closely.
Operational feasibility	No	No change required. Confirm actual device and internet access with the
		stakeholder/users during the development.
Economic feasibility	No	No change required. Keep development resources within the available tools already identified.
Schedule feasibility	Yes	Control scope by simplifying or postponing non-essential functions and reserve dedicated time in the project schedule for integration testing and stakeholder feedback.
Legal feasibility	No	No requirement change. Reinforce NFR2 and NFR5 by keeping credentials/private keys out of GitHub and using test/development data.
Ethical Feasibility	No	No change required. Continue limiting data collection to information needed for the accommodation enquiry and viewing process.

High Project risks	Yes	Add the mitigation actions from the Project Risks section below(early Supabase integration testing, scope control, prioritizing must have requirements, access restrictions) to the project schedule and development plan.

Project Risks

A detailed risk register is provided in the document Easy_Rentals_Risk_Register.xlsx. The twelve risks have been identified: 5 Critical, 6 High, and 1 Medium priority, all currently active. The table below summarizes each risk alongside it's mitigation strategy.

Risk Id	Risk Name	Priority	Mitigation Notes	Owner
Risk 1	Mobile–Backend Integration Failure	Critical	Create the Supabase connection early; test authentication, read/write operations and error handling before building all mobile features.	Technical Lead
Risk 2	Mobile and Web Data Not Synchronized	Critical	Use Supabase as the shared source of data; define table fields and validation clearly; test end-to-end data flow between Flutter, Supabase and ASP.NET.	Integration Lead
Risk 3	Web–Backend Integration	Critical	Build and test a small ASP.NET-toSupabase proof of connection first. Test CRUD operations and document configuration.	Web Developer

Risk 4	Scope Creep	Critical	Keep the agreed exclusions visible; assess every new request for scope, time and technical impact; record approved changes in the Requirements Change Log.	Project Manager
Risk 5	Insufficient Development and Testing Time	Critical	Prioritize Musthave requirements, develop the core end-to-end flow first, monitor the schedule weekly and reserve time for testing and corrections.	Project Manager
Risk 6	Late Stakeholder Requirement Changes	High	Validate the refined SRS and important	Project Manager

			screen/database decisions early; record all changes and assess their impact before accepting them.	
Risk 7	Excessive Data Collection	High	Collect only information required by the approved requirements; use test/development data where possible; review database fields before implementation.	Project Manager/ Backend Lead

Risk 8	Unauthorized Access to Student Information	High	Separate student and Easy Rentals management permissions; restrict management functions to authorized users; test access rules before demonstration.	Backend Lead
Risk 9	Uneven Team Contribution	High	Assign clear owners in Microsoft Project; update the Progress Tracker and individual contribution records; review blockers regularly.	Project Manager
Risk 10	Database Design Changes Cause Rework	Medium	Finalize the core database structure before major coding; review relationships for properties, students, enquiries and viewing requests;	
			test with sample data.	
Risk 11	Loss or Corruption of Project Data	High	Use proper database validation, test with sample data during development, restrict database modification permissions, maintain versioncontrolled code and regularly verify test data can be recovered.	Database Lead/Backend

Risk 12	Integration and Testing Defects Found Late	High	Perform continuous integration testing; test the core student-toSupabase-to-web flow early; maintain a defect list and retest fixes.	Testing/ Integration Lead

Risk management focus for the next phase: (1) establish the Supabase database and prove Flutter/ASP.NET connectivity early; (2) protect student information and restrict management access; (3) build the core end-to-end enquiry/viewing workflow before optional features; and (4) update the register whenever a risk changes or a new risk is identified.

Requirement Change Log

Requirements affected by the feasibility study or risk analysis are recorded below. Where a requirement was removed or postponed, this log documents why so the decision is not simply deleted without explanation.

Requirement ID	Original Requirement	Change Made	Reason for Change
FR7	The web application must allow Easy Rentals to view and manage accommodation viewing requests	Simplified to basic date/time selection with supporting notes.	Technical feasibility – calendar integration requires additional development skills and time the group does not currently have.
	(originally intended to include full calendar scheduling integration).		
N/A	Real-time notifications (considered during planning, not a formal SRS requirement).	Postponed as a future enhancement; simple in-app notifications only for the prototype.	Schedule feasibility – not enough remaining time to implement a complex real-time notification system.
N/A	Advanced admin dashboard analytics (considered during planning, not a formal SRS requirement).	Removed from the current scope	Economic feasibility – not essential to the prototype and would increase scope beyond what is required.

N/A	Full rental life cycle management (rent collection, maintenance management, payment management).	Already excluded; exclusion reconfirmed.	Project exclusions – confirmed outside scope by the feasibility study; no change needed.

Refined Project Scope

Subsequent to the feasibility study, risk review, and requirement change log, each major requirement was evaluated and allocated one of four decisions: **Keep –** retrain as originally specified; **Simplify –** reduce complexity; **Postpone –** defer to a future release; or **Remove –** exclude from the prototype. Any decision affecting a stakeholder-requested requirement requires their confirmation prior to development.

Function or Requirement	Decision	Reason	Stakeholder Informed
View accommodation information (FR1)	Keep	Essential requirement – students need to see available properties.	N/A – no change
View property details (FR2)	Keep	Essential requirement – students need sufficient information before visiting.	N/A – no change
Submit accommodation enquiry (FR3)	Keep	Core requirement – replaces manual WhatsApp enquiries.	N/A – no change
Submit viewing request (FR4)	Keep	Core requirement – replaces manual scheduling.	N/A – no change
View enquiries (FR5)	Keep	Essential for Easy Rentals to manage student requests.	N/A – no change
Manage property information (FR6)	Keep	Essential for Easy Rentals to maintain accurate listings.	N/A – no change
Manage viewing requests (FR7)	Simplify	Full calendar scheduling is too complex; use basic date/time selection instead.	To be confirmed Inform the stakeholder before development begins
Store student information (FR8)	Keep	Necessary for the enquiry process.	N/A – no change
Store accommodation information (FR9)	Keep	Necessary for property listings.	N/A – no change

Share information between apps (FR10)	Keep	Essential for system integration.	N/A – no change
Authorized access only (NFR1)	Keep	Essential for security.	N/A – no change
Protect student information (NFR2)	Keep	Essential for privacy.	N/A – no change
Easy navigation (NFR3)	Keep	Important for user experience.	N/A – no change
Clear error messages (NFR4)	Keep	Important for usability.	N/A – no change
Use test data (NFR5)	Keep	Required by project guidelines.	N/A – no change
Consistent information (NFR6)	Keep	Essential for reliability	N/A – no change
Advanced calendar/scheduling	Postpone	Too complex for the prototype; implement basic date/time selection now	To be confirmed – flag as enhancement in the next stakeholder update
Real-time notifications	Postpone	Too complex for the prototype; implement simple in-app notifications now.	To be confirmed - flag as enhancement in the next stakeholder update

This refined scope carries forward into the next development phase. The SRS therefore now reflects the development path: **stakeholder problem – requirements – feasibility – risks – refined project scope.**

PRIORITISE THE REQUIREMENTS

ID	REQUIREMENT	TYPE	PRIORITY	RELATED STAKEHOLDER NEED
FR1	The system must allow prospective students to view available	Functional	Must have	Students need access to accommodation information
	accommodation information.			before deciding whether to visit.

FR2	The system must allow prospective students to view property details, including address, number of rooms, accommodation rules, deposit price, facilities and available/moving-in date.	Functional	Must have	Students need sufficient property information to determine whether the accommodation is suitable.
FR3	The mobile application must allow prospective students to submit an accommodation enquiry.	Functional	Must have	Students currently make accommodation enquiries through individual communication with Easy Rentals.
FR4	The mobile application must allow prospective students to submit an accommodation viewing request.	Functional	Must have	Students currently need to communicate with Easy Rentals to arrange physical viewings.
FR5	The web application must allow Easy Rentals to view prospective student enquiries.	Functional	Must have	Easy Rentals needs a more organised way of managing prospective student enquiries
FR6	The web application must allow Easy Rentals to manage accommodation/property information	Functional	Must have	Easy Rentals needs to provide students with relevant and up-to-date property information.
FR7	The web application must allow Easy Rentals to view and manage accommodation viewing requests using basic date/time selection (simplified from full calendar scheduling).	Functional	Must have	Easy Rentals needs to manage viewing arrangements more efficiently and reduce scheduling difficulties
FR8	The system must store relevant prospective student information required for the accommodation enquiry process.	Functional	Must have	Easy Rentals currently collects information about prospective students during the enquiry process.
FR9	The system must store relevant accommodation information required by	Functional	Must have	Students need access to important accommodation and property information.

	prospective students.			
FR10	The system must allow relevant accommodation and enquiry information to be shared between the mobile and web applications.	Functional	Must have	Students and Easy Rentals need access to relevant shared information.
NFR1	Only authorized Easy Rentals users must be able to access or modify management information through the web application.	Non-functional	Must have	Easy Rentals needs controlled access to accommodation and student information.
NFR2	The system must protect prospective student information from unauthorized access.	Non-functional	Must have	Student information must be handled securely.
NFR3	The mobile application must be understandable and easy for prospective students to navigate	Non-functional	Should have	Students need a convenient and accessible way to obtain accommodation information.
NFR4	Error messages displayed to users must be clear and understandable.	Non-functional	Should have	Users need to understand problems when an action cannot be completed.
NFR5	The prototype must use test/development data where real confidential information is not required.	Non-functional	Must have	The project must avoid unnecessary exposure of real student/stakeholder information during development.
NFR6	The system must maintain consistent accommodation and enquiry information between the mobile and web applications	Non-functional	Must have	Both students and Easy Rentals need reliable and consistent information.
SR1	The mobile application must be developed using Flutter.	System	Must have	The project requires a Flutter mobile application.
SR2	The web application must be developed using ASP.NET.	System	Must have	The project requires an ASP.NET web application

SR3	The shared backend/database must use Supabase.	System	Must have	The mobile and web applications require a shared backend/database.

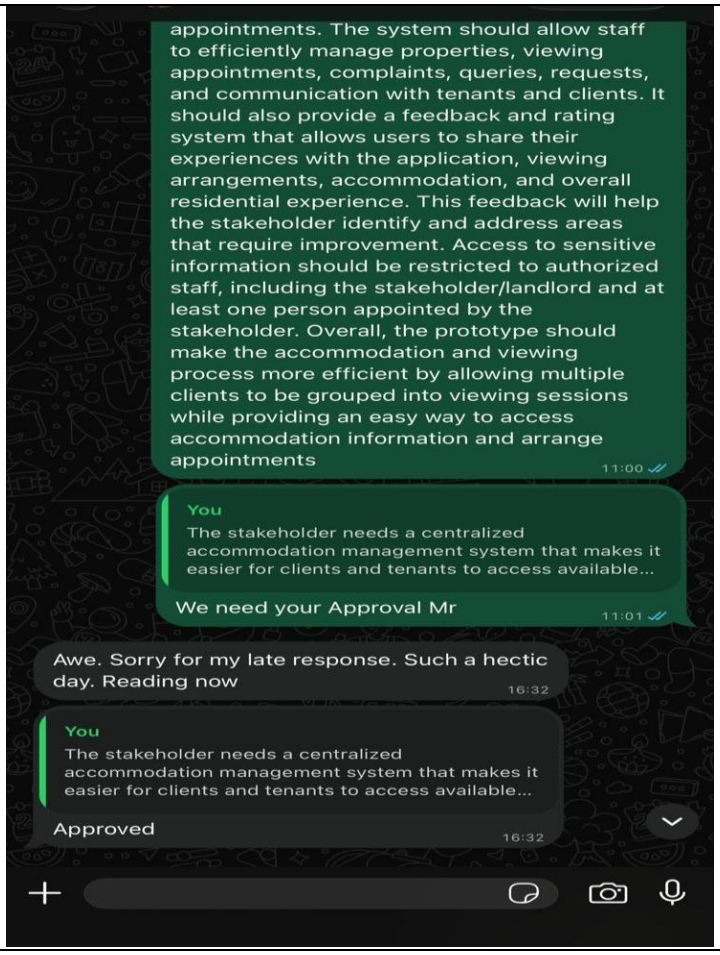
SR4	The project's source code and relevant documentation must be maintained using GitHub.	System	Must have	The project requires version control and collaborative development/documentation.
SR5	The system must be developed and tested using appropriate development environments for Flutter, ASP.NET and Supabase.	System	Must have	The development team needs suitable environments to build and test the prototype.
IR1	The Flutter mobile application must communicate with the Supabase backend to retrieve relevant accommodation information.	Integration	Must have	Prospective students need to access accommodation information through the mobile application.
IR2	The Flutter mobile application must send prospective student enquiries and viewing requests to the Supabase backend.	Integration	Must have	Student enquiries and viewing requests need to reach Easy Rentals.
IR3	The ASP.NET web application must retrieve relevant accommodation, enquiry and viewing information from Supabase.	Integration	Must have	Easy Rentals needs access to information submitted or stored in the system.
IR4	Information submitted through the mobile application must be available to authorized Easy Rentals users through the web application.	Integration	Must have	Easy Rentals needs to receive and manage student enquiries and requests.

IR5	Accommodation information updated through the web application must be reflected in the relevant information available through the mobile application.	Integration	Must have	Students need access to current accommodation information.
IR6	The mobile application and web application must use the Supabase backend as the shared source of relevant project data.	Integration	Must have	The proposed system requires a shared source of information between its components.

ADD SUPPORTING EVIDENCE

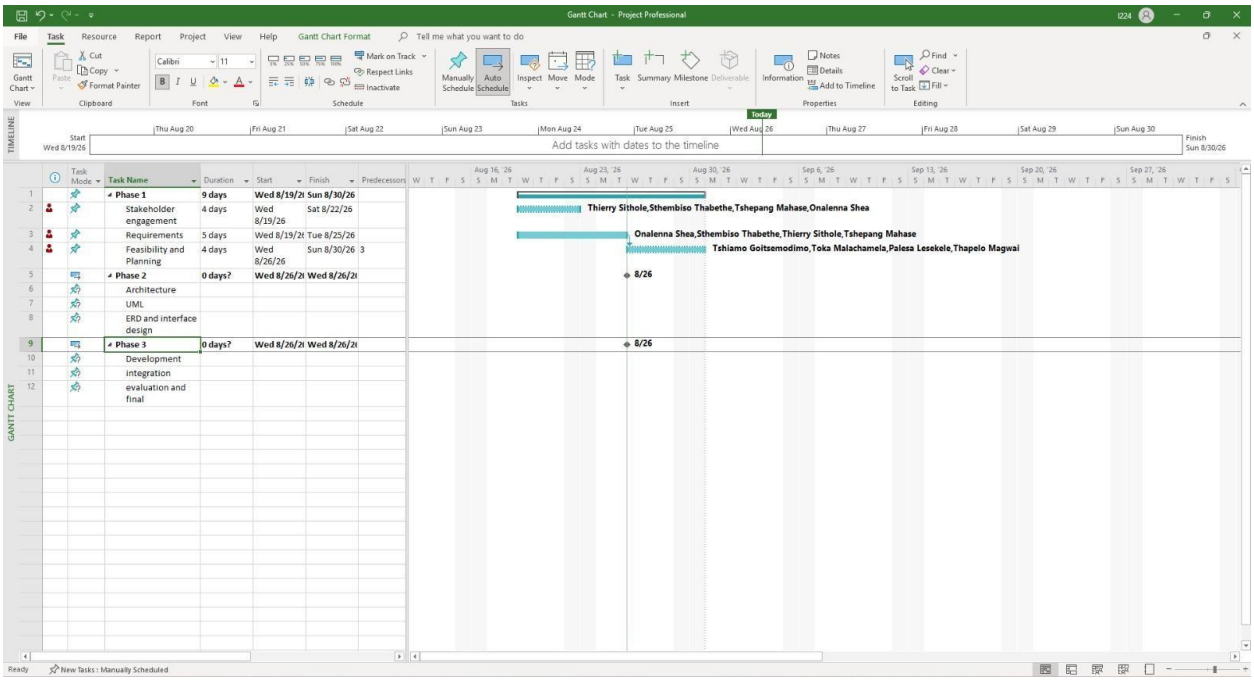
Appendix	Evidence
Appendix A	1. What is happening now? 2. What problems are experienced? 3. Who experiences the problem? 4. What are the consequences of the problem? 5. Why is an improved solution required? 6. What information is used? 7. What should users be able to do using the mobile application? 8. What should staff be able to manage using the web application? 9. What reports or summaries are needed from the users? 10. Who should be allowed to view or change information? 11. How will you know that the proposed prototype addresses the problem?

Appendix B	1. Easy Rentals manages accommodation enquiries and viewings mainly through WhatsApp and individual communication. 2. The process is time-consuming, costly and difficult to coordinate, especially when arranging individual viewings. 3. Easy Rentals, prospective students and current tenants. 4. Wasted time, increased travel/petrol costs, administrative workload and disruption to current tenants. 5. To provide accommodation information digitally and reduce unnecessary physical viewings. 6. Student details, property details, accommodation rules, deposit price, facilities and available/moving-in dates. 7. View accommodation information, submit enquiries and request accommodation viewings. 8. Manage property information, student enquiries and viewing requests. 9. Reports or summaries of accommodation enquiries, viewing requests and available properties. 10. Easy Rentals staff should manage information, while prospective students should access relevant accommodation information. 11. If students can access information before visiting and Easy Rentals can manage enquiries and viewing requests more efficiently.
Appendix C	Here's the supporting meeting link we had with the stakeholder: https://teams.microsoft.com/meet/310639985206948?p=vz2Nn8Ae6mw0D0Cb4f

Appendix D	

Appendix E	

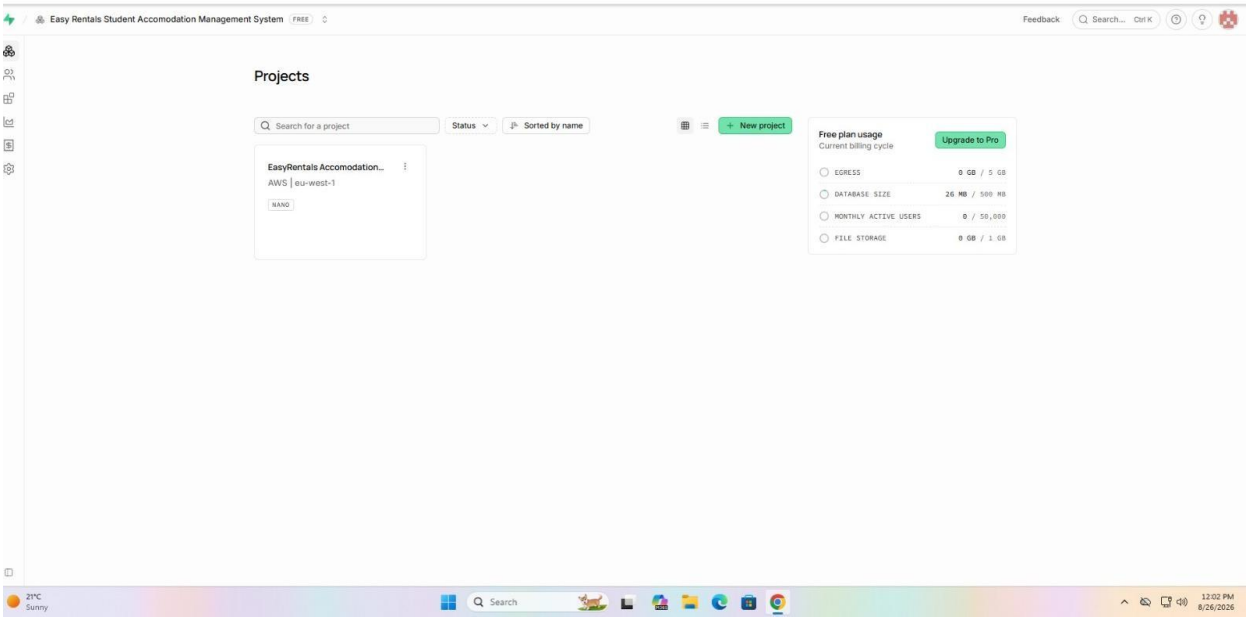
Microsoft Project Schedule

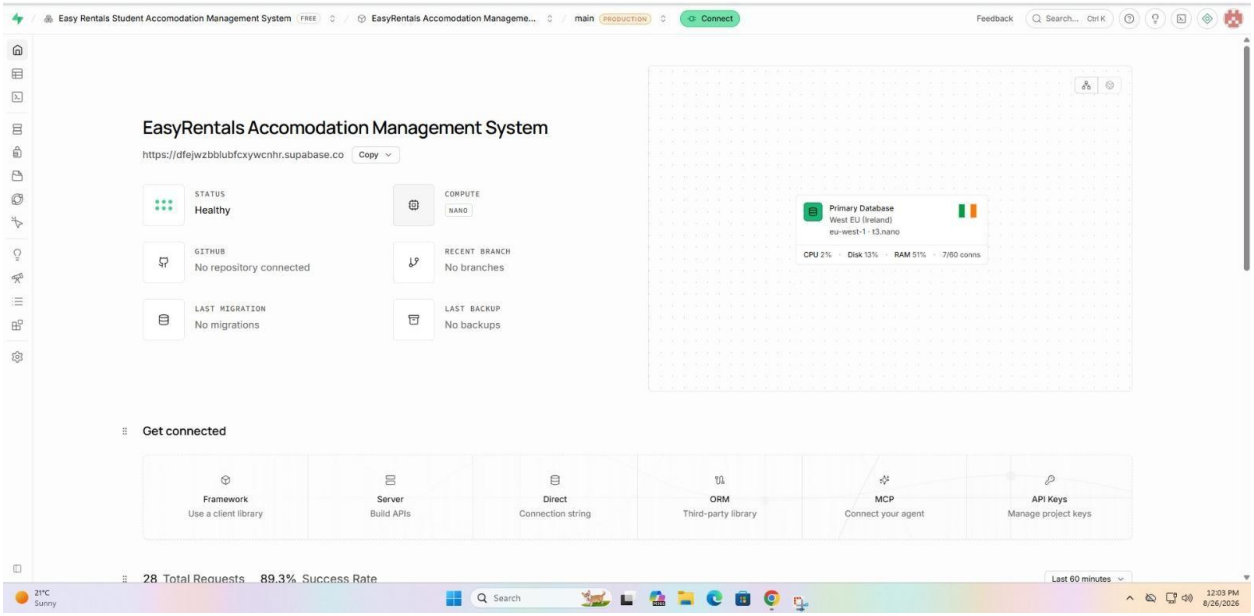


Github Repository Link

https://github.com/Theo-debugg/Easy-Rentals-Accommodation-management-system-.git

Supabase Project





Phase 2

Designing System Architecture Main Components

Component	Purpose
Flutter Mobile Application	Provides mobile functionality for the prospective students
ASP.NET Web Application	Provides web portal for the intended Easy Rentals staff or administrative users
Supabase Authentication	Manages authentication where required
Supabase Database	Stores student and staff profiles, property listings, enquires, viewing requests and feedbacks
Supabase Storage	Stores files or documents where required

Who Uses Each Application

User/Role	Uses	Main Purpose
Prospective Student	Flutter	View accommodation information, view property details, make appointment, check appointment status, send feedback and enquiries
Easy Rentals Staff	ASP.NET	Manage accommodation information, view and manage prospective students enquiries, manage appointment/viewing related information, view student information

Responsibilities of Each Component of the Designing System Architecture

Components	Responsibilities
Flutter	Allow Prospective students to: sign up, sign-in, view accommodation information, view property details, make appointment, check appointment status, send feedback and enquiries
ASP.NET	Allow Easy Rentals staff to: sign-in, manage accommodation information, view and manage prospective students enquiries, manage appointment/viewing related information, view student information
Supabase Authentication	Verify prospective students, staff identities and manages student and staff accounts.
Supabase Database	Stores student and staff profiles, property listings, enquires, viewing requests and feedbacks
Supabase Storage	Stores PDF files and all related media files involved

Main Data Flows of the Designing System Architecture

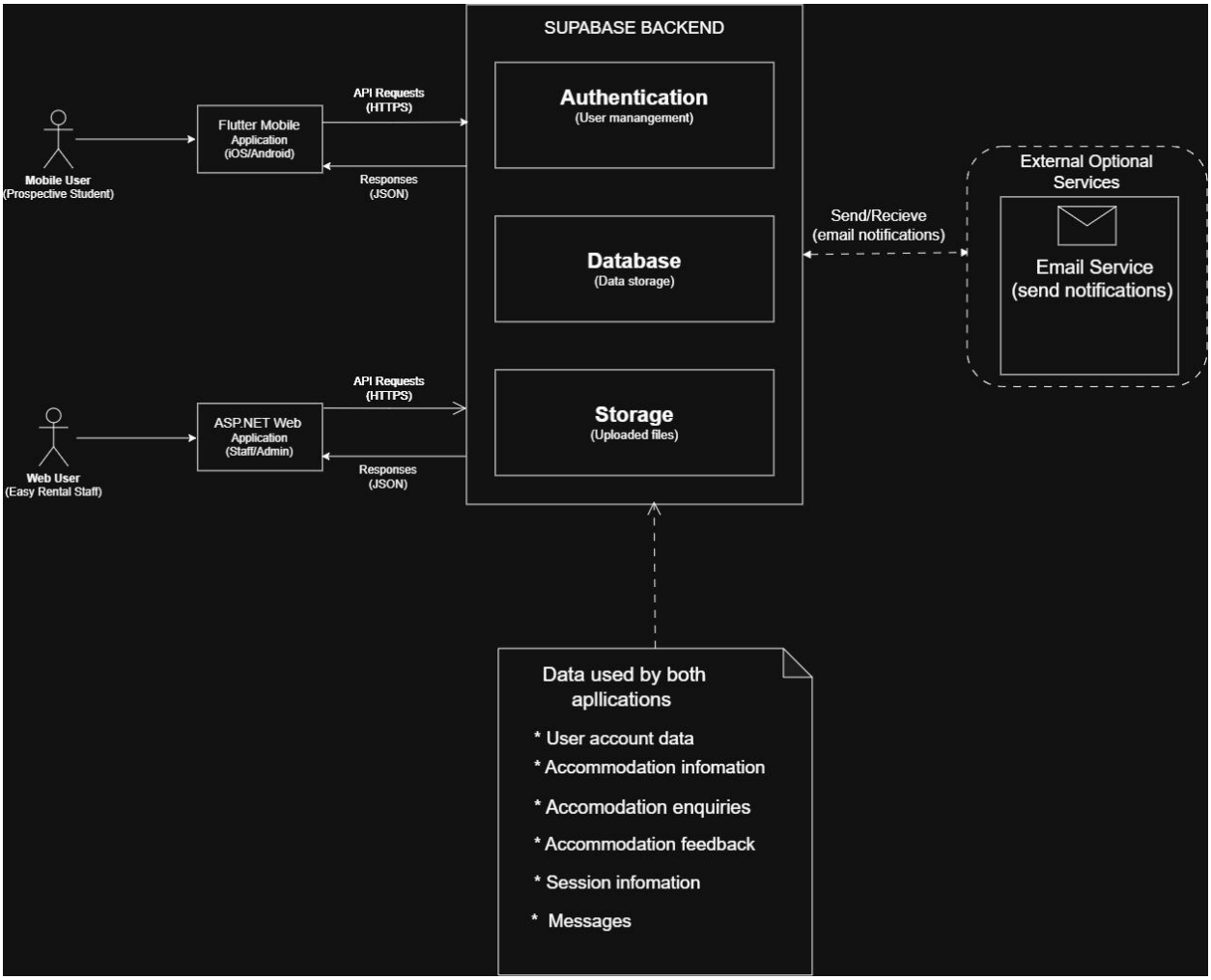
From	To	Info exchanged
Student	Flutter App	Student enters registration details such as name, email, and password to create an account.
Flutter App	Supabase	New student account information is sent to Supabase for secure storage.
Student	Flutter App	Student submits a property viewing request with the property ID and preferred date/time, or enters an inquiry message.
Flutter App	Supabase	Viewing request or inquiry information is sent to Supabase and stored.
Supabase	ASP.NET Staff App	New viewing requests and inquiries are retrieved by the staff web application.
ASP.NET Staff App	Staff	Staff can view the student's details, contact information, property details,

		and inquiry/request.
Staff	ASP.NET Staff App	Staff enters a response or updates the viewing request status, such as Confirmed or Cancelled.
ASP.NET Staff App	Supabase	Staff responses and updated viewing statuses are sent to Supabase and saved.
Supabase	Flutter App	Updated viewing status or staff response is retrieved by the student's application
Flutter App	Student	Student receives a notification and can view the staff response or the confirmation/cancellation of the viewing request.

System Architecture Diagram link

https://lucid.app/lucidchart/d8c7306c-da9c-4b2e-a34d-2d6b52defdb1/edit?viewport_loc=-118%2C-102%2C2395%2C1245%2C0_0&invitationId=inv_118af968-481f-4907-9049-6fcb8e7fdecd

System Architecture Diagram



Explanation of the System Architecture

The Flutter mobile application is used by prospective students to view accommodation information, submit enquiries, and request accommodation viewings. The ASP.NET web application is used by Easy Rentals staff to manage accommodation information, enquiries, and viewing requests.

Supabase acts as the shared backend for both applications. It provides authentication, database storage, and file storage. The database stores information such as user accounts, accommodation details, enquiries, feedback, and viewing information.

The Flutter application communicates with Supabase through HTTPS API requests to retrieve information and submit student enquiries and viewing requests. The ASP.NET application also communicates with Supabase through HTTPS API requests to retrieve and manage accommodation, enquiry, and viewing information.

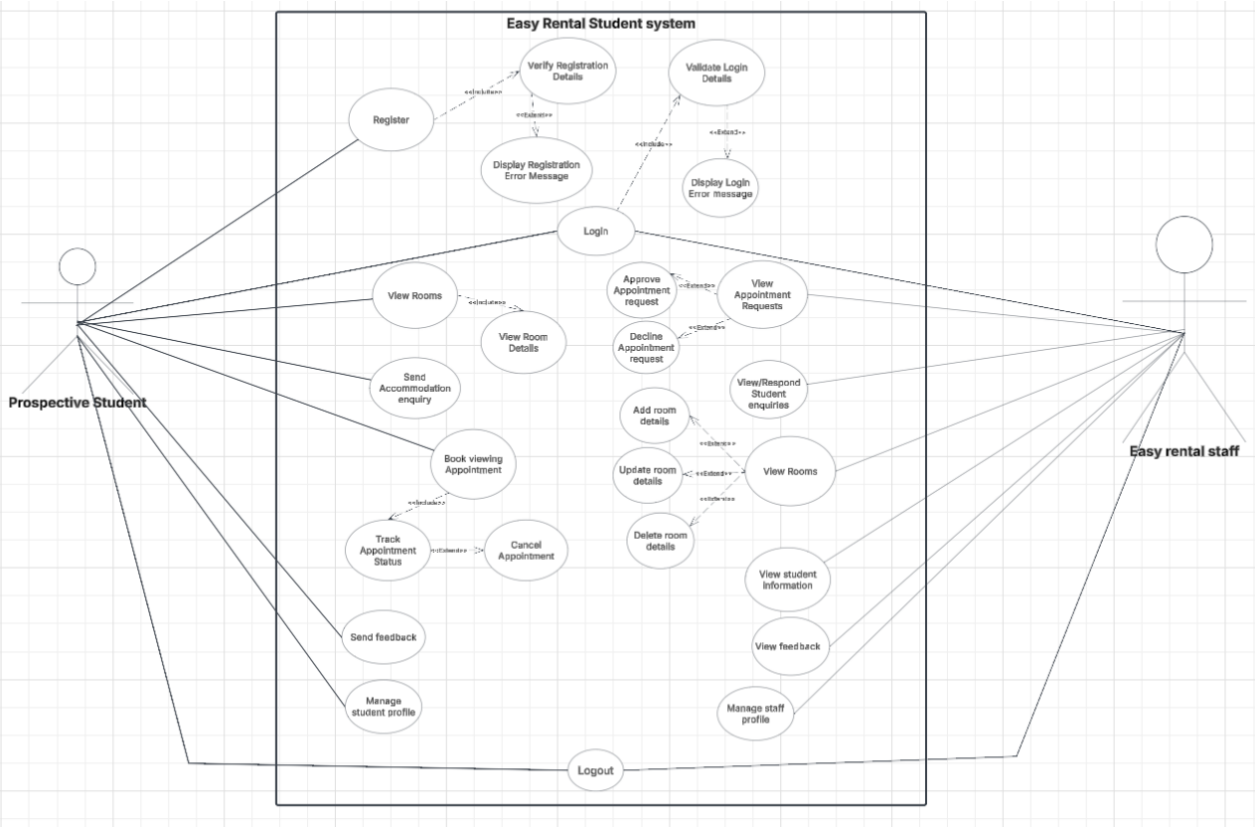
Identifying the Actors

Actor	What Does This Actor Need To Do?
Student	Register, login, view accommodation info, submit enquiries, request viewing, send feedback
Easy Rental Staff	Login, upkeep property info, respond to enquiries, manage viewing requests, view feedback

Use case Diagram link

https://lucid.app/lucidchart/2b063380-1ef2-478a-aeb2-bfa37c88595b/edit?viewport_loc=-951%2C-915%2C2637%2C1818%2C0_0&invitationId=inv_a872a4e7-3bd8-474d-a1f9-dabc69af82b7

Use case Diagram picture



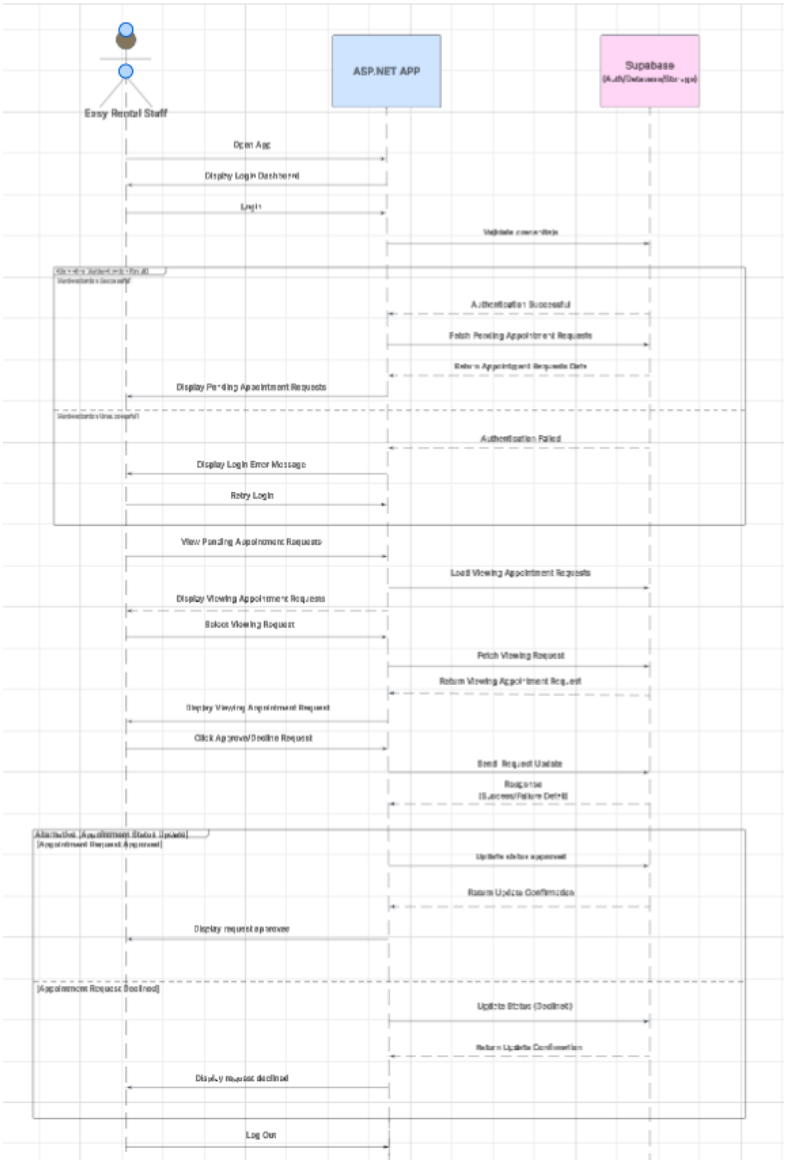
Participants of the ASP.NET sequence diagram

Participant	Role
Easy Rental Staff	The staff logs in, views pending viewing requests, and approves or declines them.
ASP.NET App	The web application that handles the login, shows the dashboard and requests, and sends status updates to Supabase.
Supabase (Auth/Database/Storage)	The backend that verifies credentials, stores and returns viewing request data, and saves approved or declined statuses.

ASP.NET sequence diagram link

https://lucid.app/lucidchart/42326f7e-e6e0-44f5-96fe-e8685c29f9b8/edit?viewport_loc=-2141%2C-4199%2C5833%2C3960%2C0_0&invitationId=inv_bc6f1276-4c17-4035-a3e4-372034c6bb37
ASP.NET sequence diagram

ASP.NET sequence diagram picture



ASP.NET sequence diagram against Use case

Use Case	Possible Sequence Diagram
View Pending Appointment Requests	Staff → ASP.NET → Supabase
View student information	Staff → ASP.NET → Supabase
Approve/Decline Requests	Staff → ASP.NET → Supabase
Add/Update/Delete room details	Staff → ASP.NET → Supabase

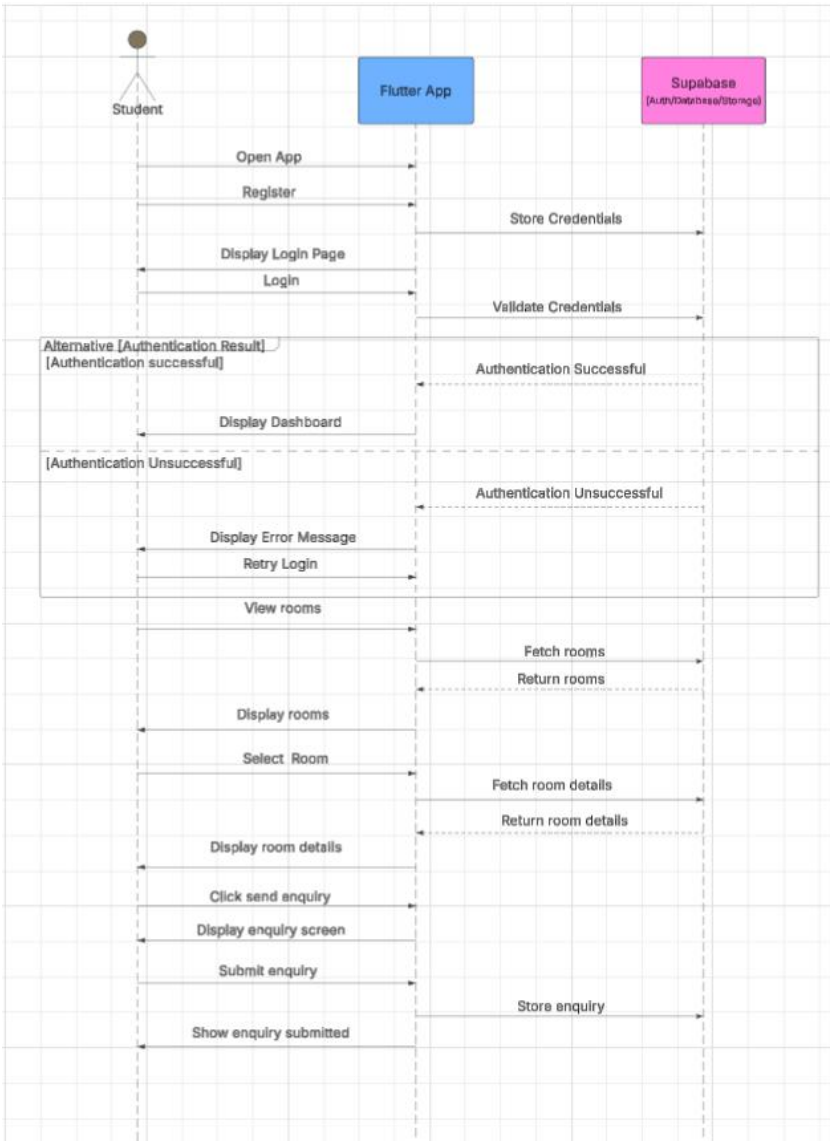
Participants of the Flutter sequence diagram

Participant	Role
Student	The student Register and logs in, View property details, Send Accommodation enquiry, Request accommodation viewing, Check status/cancel status and send feedback.
Flutter App	The Flutter application receives the requests and sent information to supabase.
Supabase (Auth/Database/Storage)	The backend that verifies credentials, stores the enquiries and the requests and retrieves the property information.

Flutter sequence diagram link

https://lucid.app/lucidchart/8de32e4d-911c-431b-a35b-127f77eed3db/edit?invitationId=inv_8a53e061-7771-4c2f-b72c-a4b1d28faedc&page=0_0#

Flutter sequence diagram picture



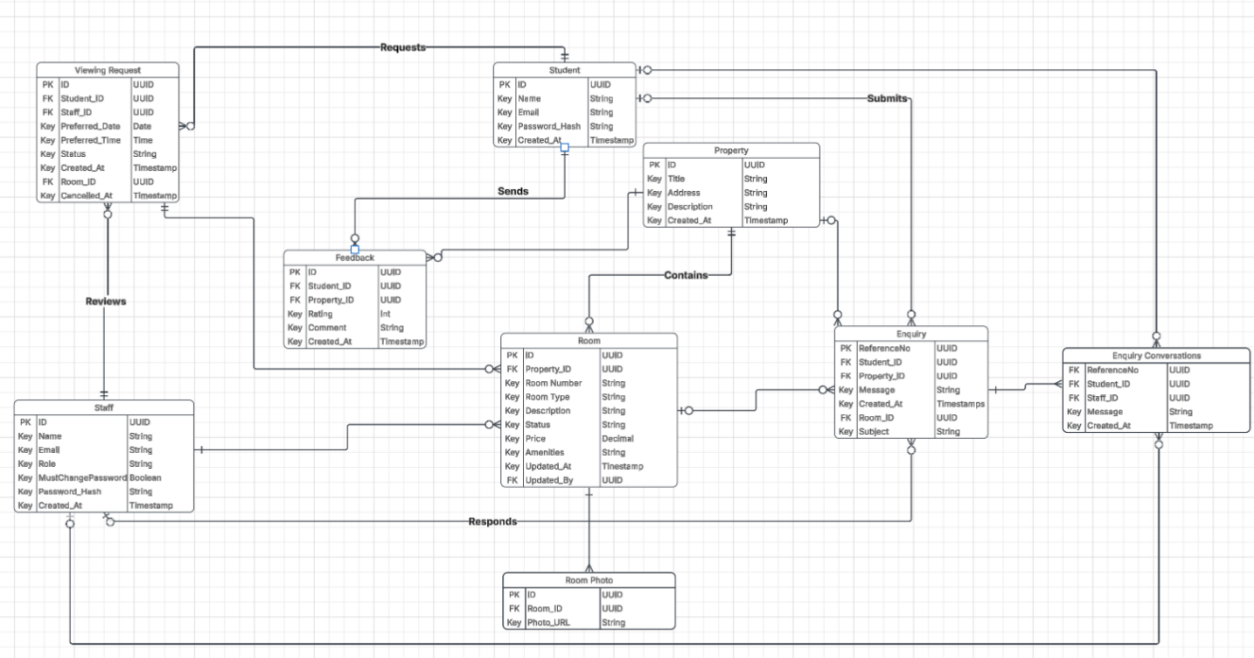
Flutter sequence diagram against Use case

Use Case	Possible Sequence Diagram
View Rooms	Student → Flutter → Supabase
Send Accommodation Enquiry	Student → Flutter → Supabase
Track appointment status	Student → Flutter → Supabase
Manage student profile	Student → Flutter → Supabase

ERD link

https://lucid.app/lucidchart/76039561-d095-45b5-856f-46c3432533c2/edit?viewport_loc=-1405%2C-484%2C3528%2C1445%2C0_0&invitationId=inv_123df1f3-67c9-4362-b0fd-3d8b4863f2c5

ERD picture



Review of Approved Requirements

Student (Flutter mobile app)

Requirement/ Use Case	Possible Interface
Student must register for an account	Sign up screen
Student must log in	Login screen
Student must view available rooms	Dashboard → Properties screen
Student must view a room's details	Room details screen
Student must send an accommodation enquiry	Send Enquiry screen and Enquiry Conversation screen
Student must book a viewing	Book Appointment Viewing screen
Student must view appointment status	My Requests screen
Student must cancel an appointment	My Request screen – Cancel button
Student must give feedback	Give Your Feedback screen
Student must manage their profile	Profile screen

Easy Rentals Staff (ASP.NET web app)

Requirement/ Use Case	Possible Interface
Staff must log in	Staff Portal login page
Staff must view appointment requests	Viewing Requests page
Staff must approve or decline requests	Viewing Requests page
Staff must view and respond to enquiries	Enquiries page
Staff must view rooms	Room Dashboard page
Staff must add, update and delete rooms	Room Dashboard
Staff must view student information	Student details on the Viewing Requests and Enquiries pages
Staff must view feedback	Feedback page
Staff must manage their profile	Staff Profile page with Set your password
Staff must log out	Sidebar – Logout link

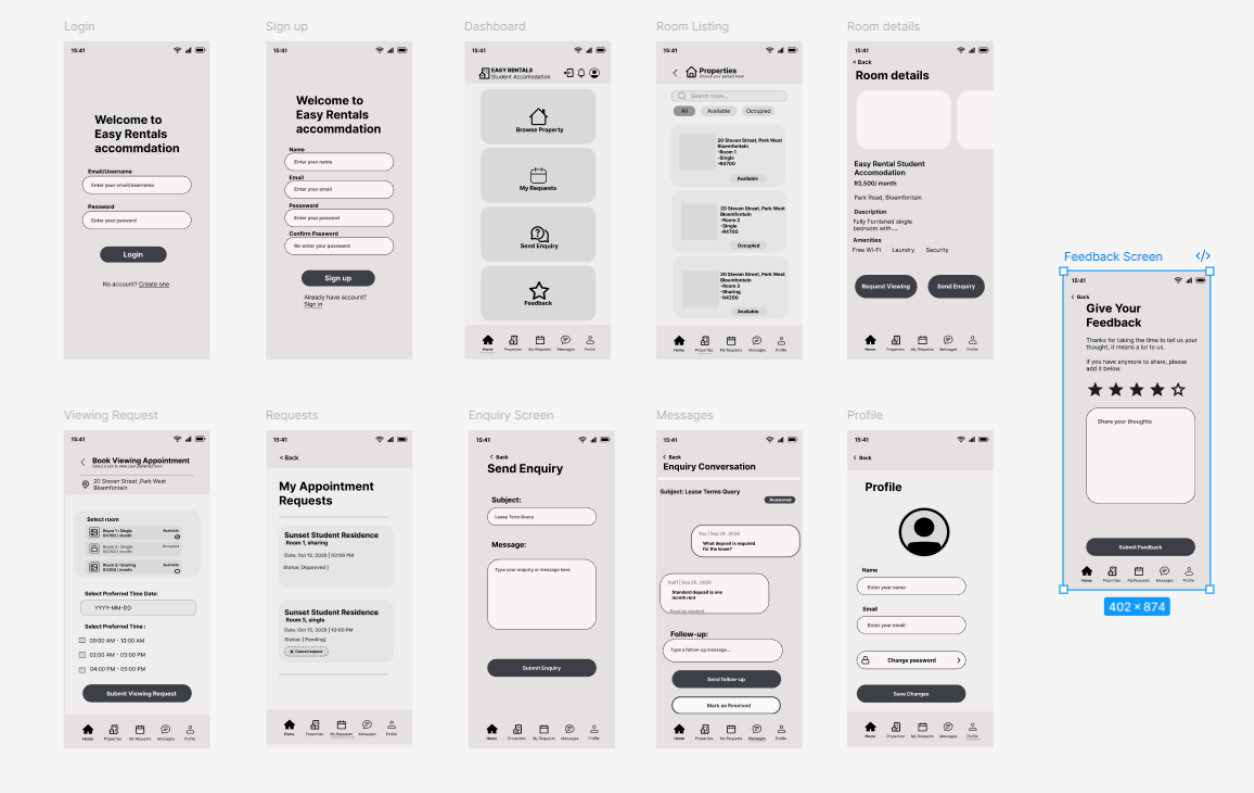
Identification of Flutter Screens

Flutter Screens	User	Purpose	Use Case
Login	Student	Authenticate Student	Login
Sign Up	Student	Register Student Profile	Register
Dashboard	Student	Summary Card of what the student can do on the application	
Room Listing	Student	List of the rooms the student can choose from	View Rooms
Room details	Student	Contains the details of an individual room	View Room details
Enquiry Screen	Student	Allow student to send an enquiry	Send Accommodation Enquiry
Feedback Screen	Student	Allow the student to rate a room	Send feedback
Profile Screen	Student	Allow the student to edit personal details	Manage Student profile
Requests Screen	Student	View the 'Viewing' appointments	Track Viewing appointment status
Viewing Request Screen	Student	Book a Viewing appointment	Book Viewing appointment
Message Screen	Student	Enquiry conversation	Send Accommodation Enquiry

Flutter Screens link

https://www.figma.com/design/f1h48hYRyC20CkUCM3jJfT/Flutter-mobile-app?node-id=1-2&t=zp0S1GrUdkLA9c4v-0

Flutter Screens Picture



Identification of ASP.NET Screens

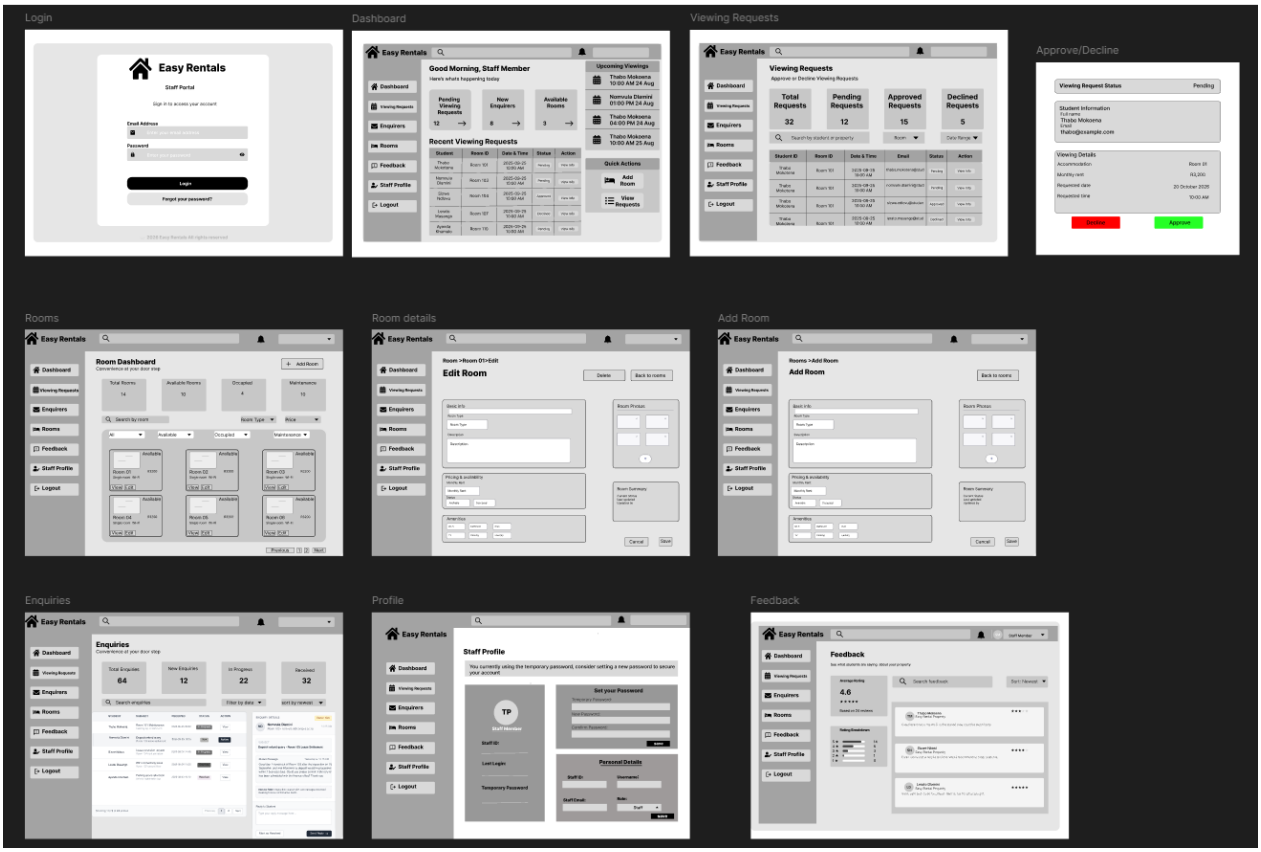
Flutter Screens	User	Purpose	Use Case
Login	Staff	Authenticate the Easy Rentals staff member	Login
Dashboard	Staff	View a summary of accommodation, enquiries and viewing requests	
Viewing Requests	Staff	View student accommodation viewing requests.	View Appointment Requests

Enquiries	Staff	View accommodation enquiries submitted by prospective students	View/Respond Student enquiries
Approve/Decline Request	Staff	Easy Rental staff must be able to approve and decline viewing requsts	Approve Decline Viewing Requests
Rooms	Staff	View and manage available accommodation and room information	
Add Room	Staff	Add Room to the Accomodation	Add/Update/Delete property details
Edit Room	Staff	View and update detailed accommodation information such as rooms, rules, deposit, facilities and availability	Add/Update/Delete room details
Feedback	Staff	View feedback submitted by students regarding accommodation	View feedback
Staff profile	Staff	View and update staff account/profile information and password	Manage staff profile
Logout	Staff	Logs the current EasyRental staff out	Logout

ASP.NET Screens link

https://www.figma.com/design/4QkvwEdkyLsApZxUQzr2T2/ASP.NET-WIREFRAMES?node-id=0-1&t=L4PTZolKGSiFaP98-1

ASP.NET Screens picture



ASP.NET Screens Navigation Flow

Starting Screen	User Action	Next Screen
Staff Sign In	Enter credentials and click [Sign In]	Dashboard
Dashboard	Click Pending Viewing Requests card	Viewing Requests (List)
Dashboard	Click [Add Room] (Quick Action)	Edit Room (Add Mode)
Dashboard	Click New Enquirers card	Enquiries (List)
Dashboard	Click [View Requests] (Recent Viewing Requests)	Viewing Requests (List)
Viewing Requests	Click [View Request] on a specific request	Viewing Request Details
Viewing Request Details	Click [Confirm Viewing] or [Cancel Viewing]	Viewing Requests (Updated Status)
Enquiries	Click [View] on a specific enquiry	Enquiry Details / Response
Enquiry Details / Response	Type response and click [Send Response]	Enquiries (Updated Status)
Rooms	Click [Edit] on a room card	Edit Room
Edit Room	Click [Save] or [Cancel]	Rooms

Feedback	Click [Reply] on a review	Feedback Reply (Modal/Inline)
Staff Profile	Click [Change Password]	Set your Password
Staff Profile	Click [Edit Profile]	Personal Details
Staff Profile	Click [Sign Out]	Staff Sign In

Flutter Screen Navigation Flow

Starting Screen	User Action	Next Screen
Login	User clicks “No account? Create one”	Register(Screen)
Register	User fills in Username, Email, Password, Confirm Password and clicks “Sign Up” button	Dashboard(Screen)
Register	User clicks “Already have account? Sign in”	Login(Screen)
Login	User enters credentials and clicks “login” button	Dashboard(Screen)
Dashboard(Home)	User clicks on browse rooms	Room listing(Screen)
Room listing	User clicks on a room card	Room details(Screen)
Room details	User clicks “Request Viewing” button	Book Viewing Appointment(Screen)
Room details	User clicks “Send Enquiry” button	Send Enquiry(Screen)
Book Viewing Appointment	User selects a date/time and clicks “Submit Viewing Request” button	My Requests(Screen)
My Requests	User clicks on an appointment request (for example: “Sunset Student Residence”)	Room details(Screen)
Send Enquiry	User types a message and clicks “Submit Enquiry” button	Enquiry Conversation(Screen)
Profile(Bottom Navigation)	User changes info, and password and save changes afterwards	Home(Bottom Navigation)
Home(Bottom Navigation)	User clicks on “Feedback”	Give Your Feedback(Screen)

Mobile and Web Integration

Flutter Action	Supabase Data	ASP.NET Action
User registers(Register screen to Sign up)	User record created(full_name, email, password, role= 'student')	N/A
User logs in (Login screen to Login)	User authenticated via Supabase Authentication	Staff Logs in (Staff Sign in to Dashboard)
User views room listing(Dashboard to Browse Rooms)	Property/Room data received	Staff Manages rooms (Dashboard to Rooms to Edit Room)
User views room details(Room listing to Room card)	Property/Room details retrieved (address, rooms, rules, deposit, facilities, moving-in date)	Staff updates room info (Edit Room to Save)
User Requests viewing room(Room details to Request Viewing to Book Viewing Appointment to Submit)	Viewing Request created (student_id, preferred_date, preferred_time, status="pending")	Staff views 'Viewing appointments' (Dashboard to Pending Viewing Requests to Viewing Requests List)
User views My Requests (My Requests screen to Appointment status)	Viewing Request status retrieved (Pending /Approved /Rejected)	Staff Updates status (Viewing Request Details to Confirm Viewing or Cancel Viewing)
User sends Enquiry (Room details to Send Enquiry to Submit Enquiry)	Enquiry created (student_id, property_id, message, status="Pending ")	Staff views enquiries (Dashboard to New Enquiries to Enquiries List)
User Views Enquiry Conversation (Send Enquiry to Enquiry Conversation)	Enquiry message and status retrieved	Staff responds to enquiry (Enquiry Details /Response to Send Response)
User gives Feedback (Home to Give Your Feedback)	Feedback Created (student_id, property_id, rating, comment)	Staff views feedback (Dashboard to Feedback)
User replies to feedback thread	Feedback reply stored	Staff replies to review (Feedback to reply to Feedback Reply?)
User updates Profile (Profile to Save Changes)	User record Updated (full_name, email, password)	Staff can View student information if authorized
User views Room availability (Room listings)	Room Availability status retrieved (is_available)	Staff Updates availability (Edit Room to Save)
User Logs out (Profile to Sign Out)	Session ended	Staff logs out (Staff Profile to Sign Out)

Design against the ERD

1. Flutter Mobile App (Prospective Student)

Interface Information	Entity/Table	Attribute/Field
Email (Login)	Student	Email
Name	Student	Name

Password	Student	Password_Hash
Properties (Room List)		
Property address	Property	Address
Room number	Room	Room Number
Room type (Single / Sharing)	Room	Room Type
Room price	Room	Price
Availability status (Available / Occupied)	Room	Status
Room photo	Room Photo	Photo_URL
Room Details		
Room photos	Room Photo	Photo_URL
Accommodation name	Property	Title
Price per month	Room	Price
Address	Property	Address
Description	Room	Description
Amenities	Room	Amenities
Book Viewing Appointment		
Property address	Property	Address
Preferred date	Viewing Request	Preferred_Date
Preferred time slot	Viewing Request	Preferred_Time
Student submitting the request	Viewing Request	Student_ID
Room being viewed	Viewing Request	Room_ID
My Requests		
Property name	Property	Title
Room number and room type	Room	Room Number, Room Type
Viewing date	Viewing Request	Preferred_Date
Viewing time	Viewing Request	Preferred_Time
Request status	Viewing Request	Status
Cancel request	Viewing Request	Status, Cancelled_At
Send Enquiry		
Subject	Enquiry	Subject
Enquiry message	Enquiry	Message
Student sending the enquiry	Enquiry	Student_ID
Enquiry Conversation		
Subject	Enquiry	Subject
Enquiry status (Answered)	Enquiry	Status
Sender (Student / Staff)	Enquiry Message	Student_ID, Staff_ID
Message date	Enquiry Message	Created_At
Message text	Enquiry Message	Message
Follow-up message	Enquiry Message	Message (new record)
Profile		
Name	Student	Name
Email	Student	Email
Give Your Feedback		
Star rating	Feedback	Rating
Feedback comment	Feedback	Comment
Student giving feedback	Feedback	Student_ID

2. ASP.NET Web Portal (Easy Rentals Staff)

Interface Information	Entity/Table	Attribute/Field
Staff Login		
Email address	Staff	Email
Password	Staff	Password_Hash
Dashboard		
Student name (Recent Viewing Requests)	Student	Name
Room ID	Room	Room Number
Date & Time	Viewing Request	Preferred_Date, Preferred_Time
Status	Viewing Request	Status
Action (Approve / Decline)	Viewing Request	Status, Staff_ID
Viewing Requests		
Student name	Student	Name
Student email	Student	Email
Room ID	Room	Room Number
Date & Time	Viewing Request	Preferred_Date, Preferred_Time
Status	Viewing Request	Status
Action (Approve / Decline)	Viewing Request	Status, Staff_ID
Feedback		
Student name	Student	Name
Property name	Property	Title
Star rating	Feedback	Rating
Room		
Room image	Room Photo	Photo_URL
Room number	Room	Room Number
Room type	Room	Room Type
Amenities	Room	Amenities
Price	Room	Price
Room status	Room	Status
Description	Room	Description
Monthly rent	Room	Price

Status	Room	Status
Last updated	Room	Updated_At
Updated By	Room	Updated_By
Enquiries		
Student name	Student	Name
Subject	Enquiry	Subject
Received date	Enquiry	Created_At
Enquiry status	Enquiry	Status
Room (Enquiry Details)	Room	Room Number
Student email	Student	Email
Student message	Enquiry	Message
Message time	Enquiry Message	Created_At
Staff reply (Reply to Student)	Enquiry Message	Message, Staff_ID
Mark as Resolved	Enquiry	Status
Staff Profile		
Staff ID	Staff	ID
Staff name	Staff	Name
Staff email	Staff	Email
Role	Staff	Role
Temporary password notice	Staff	MustChangePassword
New password	Supabase Authentication	Not stored in the ERD (handled by Supabase Auth)

Alignment across the System Design

Requirement 1: Student must be able to register and log in

Artefact	**Evidence**
SRS	Functional requirement for student registration and login
Architecture Diagram	Flutter mobile application and Supabase support student authentication
Use Case Diagram	Register and Login use cases
Sequence Diagram	Student → Flutter → Supabase
ERD	Student/User information is stored
Flutter Wireframe	Registration and Login screens
ASP.NET Wireframe	Not applicable to student authentication

Requirement 2: Student must be able to view available rooms

Artefact	Evidence
SRS	Functional requirement to view available accommodation/rooms
Architecture Diagram	Flutter provides the student mobile interface and Supabase provides room/property data
Use Case Diagram	View Rooms use case
Sequence Diagram	Student → Flutter → Supabase → Flutter
ERD	Room/Property information is stored
Flutter Wireframe	Properties/Rooms screen
ASP.NET Wireframe	Staff can manage room/property information

Requirement 3: Student must be able to view room/property details

Artefact	Evidence
SRS	Functional requirement to view accommodation details
Architecture Diagram	Flutter displays property/room information retrieved from Supabase
Use Case Diagram	View Room Details use case
Sequence Diagram	Student → Flutter → Supabase
ERD	Property/Room entity and related attributes
Flutter Wireframe	Property Details screen
ASP.NET Wireframe	Staff can add, update or delete room details

Requirement 4: Student must be able to send an accommodation enquiry

Artefact	Evidence
SRS	Functional requirement for students to send accommodation enquiries
Architecture Diagram	Flutter is used to send the enquiry and Supabase stores the enquiry information
Use Case Diagram	Send Accommodation Enquiry use case
Sequence Diagram	Student → Flutter → Supabase
ERD	Enquiry/message information is stored
Flutter Wireframe	Enquiries/Messages screen and chat box
ASP.NET Wireframe	Staff can view and respond to student enquiries

Requirement 5: Student must be able to request a viewing appointment

Design Artefact	Evidence
SRS	Functional requirement for booking/requesting an accommodation viewing

Architecture Diagram	Flutter handles the student's viewing request and Supabase stores the request
Use Case Diagram	Book Viewing Appointment use case
Sequence Diagram	Student → Flutter → Supabase
ERD	Viewing Request/Appointment entity
Flutter Wireframe	Viewing Request screen
ASP.NET Wireframe	Staff can view appointment requests

Requirement 6: Staff must be able to approve or decline viewing requests

Design Artefact	**Evidence**
SRS	Functional requirement for staff to manage viewing requests
Architecture Diagram	ASP.NET provides the staff interface and Supabase stores the updated request status
Use Case Diagram	Approve/Decline Appointment Request use case
Sequence Diagram	Staff → ASP.NET → Supabase
ERD	Viewing Request/Appointment status
Flutter Wireframe	My Requests screen displays the updated status
ASP.NET Wireframe	Appointment request management screen

Requirement 7: Student must be able to track the status of a viewing request

Artefact	**Evidence**
SRS	Functional requirement to track appointment/viewing request status
Architecture Diagram	Flutter retrieves the updated viewing request status from Supabase
Use Case Diagram	Track Appointment Status use case
Sequence Diagram	Student → Flutter → Supabase → Flutter
ERD	Viewing Request/Appointment status field
Flutter Wireframe	My Requests screen
ASP.NET Wireframe	Staff can update the request status

Requirement 8: Student must be able to cancel a viewing appointment

Artefact	**Evidence**
SRS	Functional requirement to cancel an appointment
Architecture Diagram	Flutter sends the cancellation request and Supabase updates the appointment/request information
Use Case Diagram	Cancel Appointment use case
Sequence Diagram	Student → Flutter → Supabase
ERD	Viewing Request/Appointment status and request information
Flutter Wireframe	My Requests screen with cancellation action

ASP.NET Wireframe	Staff can view the updated appointment/request status

Requirement 9: Student must be able to submit feedback

Artefact	**Evidence**
SRS	Functional requirement for students to provide feedback
Architecture Diagram	Flutter is used to submit feedback and Supabase stores the feedback
Use Case Diagram	Send Feedback use case
Sequence Diagram	Student → Flutter → Supabase
ERD	Feedback entity
Flutter Wireframe	Feedback screen
ASP.NET Wireframe	Staff can view feedback

Requirement 10: Student must be able to manage their profile

Artefact	**Evidence**
SRS	Functional requirement for student profile management
Architecture Diagram	Flutter provides the student profile interface and Supabase stores the profile information
Use Case Diagram	Manage Student Profile use case
Sequence Diagram	Student → Flutter → Supabase
ERD	Student/User information
Flutter Wireframe	Student Profile screen
ASP.NET Wireframe	Staff can view relevant student information

Requirement 11: Staff must be able to manage room details

Artefact	**Evidence**
SRS	Functional requirement for staff to manage accommodation/room information
Architecture Diagram	ASP.NET provides the staff management interface and Supabase stores the room/property information
Use Case Diagram	Add/Update/Delete Room Details use case
Sequence Diagram	Staff → ASP.NET → Supabase
ERD	Property/Room entity
Flutter Wireframe	Properties/Rooms and Property Details screens display the information
ASP.NET Wireframe	Room/property management screen

Requirement 12: Staff must be able to view student information

Artefact	Evidence
SRS	Functional requirement for authorised staff to view student information
Architecture Diagram	ASP.NET provides staff access to student information retrieved from Supabase
Use Case Diagram	View Student Information use case
Sequence Diagram	Staff → ASP.NET → Supabase
ERD	Student/User entity
Flutter Wireframe	Student information originates from the student's profile
ASP.NET Wireframe	Student information management/view screen

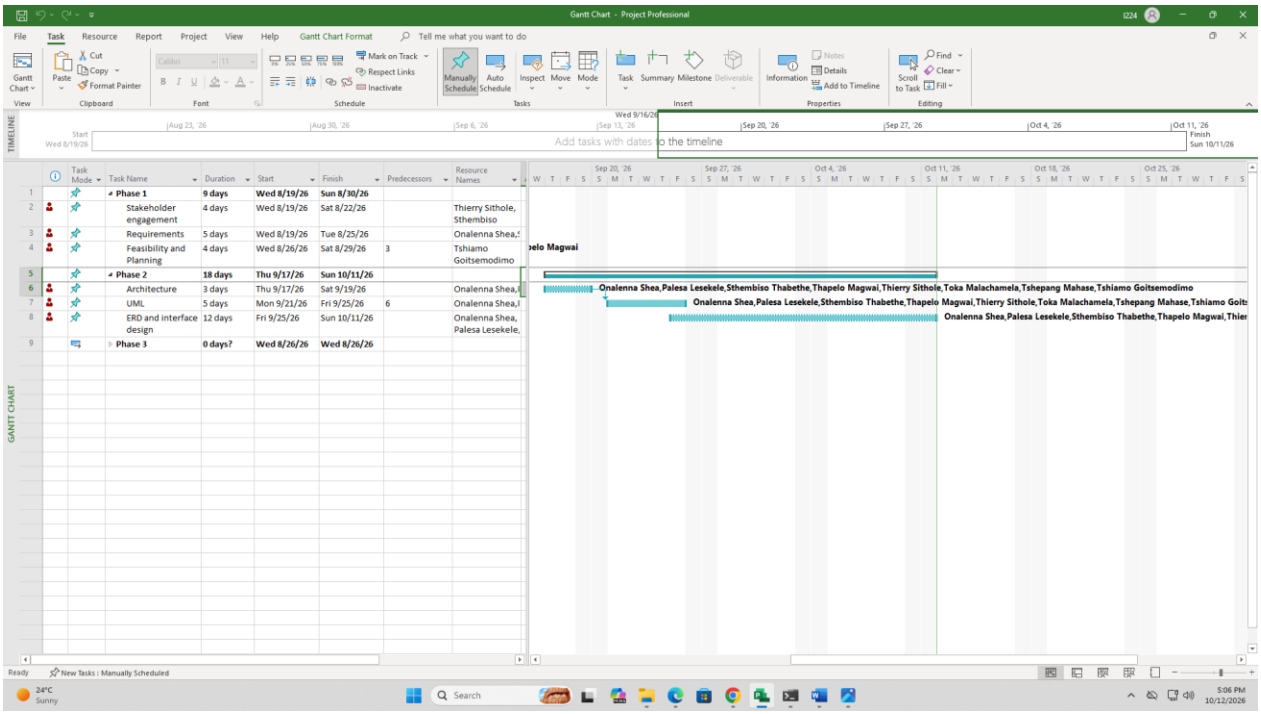
Requirement 13: Staff must be able to view feedback

Artefact	Evidence
SRS	Functional requirement for staff to view student feedback
Architecture Diagram	ASP.NET retrieves feedback information from Supabase for staff
Use Case Diagram	View Feedback use case
Sequence Diagram	Staff → ASP.NET → Supabase
ERD	Feedback entity
Flutter Wireframe	Feedback submission screen
ASP.NET Wireframe	Feedback viewing screen

Requirement 14: Users must be able to log out

Artefact	Evidence
SRS	Functional requirement for users to securely log out
Architecture Diagram	Flutter and ASP.NET provide the respective user interfaces, with authentication/session handling supported by the system architecture
Use Case Diagram	Logout use case
Sequence Diagram	User → Flutter/ASP.NET → logout process
ERD	Not directly stored as a separate entity
Flutter Wireframe	Logout option
ASP.NET Wireframe	Logout option for staff

Updated Gantt Chart



Git Hub Repository

https://github.com/Theo-debugg/Easy-Rentals-Accommodation-management-system-.git

Questions Posed to the stakeholder

- Does this screen support the way you expect the process to work?
- Is any important information missing?
- Are the labels understandable?
- Are the proposed functions suitable?
- Is the order of the process, correct?
- Is there anything that should be changed before development?

Stakeholder Feedback Evidence

